ESTRUCTURA DE DATOS · PLANTILLA DEL INFORME · C2

Tux Store v2 · Informe técnico

Evaluación por competencias C2 · NRC 42964 (lunes)

Asignatura	Estructura de Datos (24UC00428)
Docente	Carlos del Carpio
NRC · Grupo	42964 · Grupo B
Servidor	pinguino-emperador
Periodo	2026-20

Integrantes

Nombre	Usuario GitHub	Rol
FERNANDEZ RAMIREZ PATRICK MICHAEL	72672623-cmyk	Líder (scrum)
Ticona Pacco Alex Royer	Royer20061	Dev catálogo
MACHACA APAZA EDSON JAVIER	Edson-Machaca	Dev descargas
Flores Castillo Basiler Emanuel	Basilercon	Dev disco y archivo
QUISPE MOLLENIDO JUAN DIEGO	61161913-jpg	QA y documentación

Capítulo 1 · Análisis del problema

1.1 Descripción del problema [E1]

Tux Store es un simulador de consola en C++ que imita a un gestor de paquetes de Linux (como apt). Permite a los usuarios buscar, agregar, instalar y desinstalar aplicaciones en un servidor administrado (pinguino-emperador). Resuelve la gestión de un catálogo de aplicaciones usando una lista enlazada, la atención de descargas respetando prioridades

mediante una cola de prioridad, y el control del espacio en disco usando una pila que registra cada bloque instalado para poder deshacer cambios.

1.2 Variables y condiciones [E1]

Variable	Tipo en C++	Unidad	Valor inicial o rango	Condición o regla que la afecta
VELOCIDAD	const int	MB/s	6	Regla 2 (cálculo de descarga)
DISCO	const int	MB	310	Regla 4 (límite de espacio)
reloj	int	segundos	0	Regla 3 (avanza con cada atención)
libre	int	MB	310	Regla 4 y 7 (resta al instalar, suma al deshacer)
ultimaLlegada	int	segundos	0	Regla 5 (el tiempo no retrocede)
prioridad	int	entero	1, 2 o 3	Regla 6 (1 sale primero, FIFO en empates)

1.3 Requerimientos funcionales y no funcionales [E1]

ID	Requerimiento funcional	Orden(es)	Estructura
RF1	Gestionar catálogo de aplicaciones	catalogo, buscar, info, agregar	Lista enlazada
RF2	Añadir aplicaciones a la fila de descargas	instalar, fila	Cola de prioridad
RF3	Procesar descargas pendientes	procesar	Cola y Pila
RF4	Controlar el historial del disco y deshacer	historial, deshacer, espacio	Pila

ID	Requerimiento funcional	Orden(es)	Estructura
RF5	Modificar prioridades y retirar apps	prioridad, retirar	Lista enlazada
RF6	Guardar y cargar estado (Persistencia)	guardar, cargar	Archivo, Lista, Pila
RF7	Mostrar reporte y salir liberando memoria	reporte, salir	Todas

ID	Requerimiento no funcional (cómo debe comportarse)
RNF1	Ninguna entrada inválida cierra el programa.
RNF2	El código no usa contenedores estándar (list, vector, queue, stack).
RNF3	Debe liberar toda la memoria dinámica (Nodos liberados) al salir.

1.4 Estructuras propuestas y justificación [E1]

- **Catálogo (Lista Enlazada Simple):** Adecuada porque permite tamaño dinámico y la inserción/eliminación se puede dar en cualquier punto sin necesidad de memoria contigua. La búsqueda se hace recorriéndola secuencialmente.
- **Fila de Descargas (Cola de Prioridad):** Esencial porque el problema exige que las apps de prioridad 1 salgan antes que las 2, y que los empates se resuelvan por orden de llegada (FIFO). Una pila no serviría aquí (es LIFO) y una lista simple requeriría ordenar todo el tiempo.
- **Disco (Pila):** Ideal porque el requerimiento de "deshacer" (undo last) exige que el último bloque de memoria asignado sea el primero en liberarse (LIFO), emulando un historial de acciones recientes.

Capítulo 2 · Diseño de la solución

2.4 Traza a mano de la secuencia oficial [E1]

Servidor: pinguino-emperador (Velocidad: 6 MB/s, Disco: 310 MB)

#	App	Prior.	Pedida	MB	Descarga	Inicio	Fin	Espera	Libres
3	neofetc	3	0	-	-	-	-	-	-

#	App	Prior.	Pedida	MB	Descarga	Inicio	Fin	Espera	Libres
	h								
4	neovim	2	1	-	-	-	-	-	-
5	openssh	1	3	-	-	-	-	-	-
7	openssh	1	3	8	2	3	5	0	302
8	vlc	3	4	-	-	-	-	-	-
10	krita	1	8	-	-	-	-	-	-
11	firefox	2	11	-	-	-	-	-	-
13	krita	1	8	180	30	8	38	0	122
14	neovim	2	1	18	3	38	41	37	104
15	firefox	2	11	210	-	-	-	-	104 (Error: sin espacio)
16	neofetch	3	0	1	1	41	42	41	103

Fila después de la orden 6: frente -> openssh (1) -> neovim (2) -> neofetch (3) -> final
Fila después de la orden 12: frente -> krita (1) -> neovim (2) -> firefox (2) -> neofetch (3) -> vlc (3) -> final